

PARTITION ARRAY

Company: Twitter

Round: Online Assessment

Year: 2021

Difficulty: Medium

Topic: Arrays, HashMap, Frequency Counting

Source: https://algo.monster/problems/twitter_oa_partition_array

Problem Description:

You have a collection of marbles. Each marble has a number called its ID.

- If two marbles have the same ID, they are the same type.
- If their IDs are different, they are different types.

You want to give all the marbles to your friends.

There are three important rules:

1. Every marble must be given to exactly one friend.
2. Every friend must receive exactly k marbles.
3. A friend cannot receive two marbles of the same type.

There can be as many friends as needed.

The question is:

Can all the marbles be distributed while following all three rules?

Return true if it is possible; otherwise, return false.

Input Format:

The input contains:

- marbles → an array representing the type of each marble.
- k → the number of marbles that every friend must receive.

Output Format:

Return : true

if the marbles can be distributed according to the rules.

Otherwise, return : false

Sample Input:

marbles = [1, 2, 3, 4]
k = 2

Sample Output:

true

Explanation:

There are 4 marbles and each friend needs 2 marbles.

We can distribute them like this:

Friend 1 $\rightarrow$ [1, 2]
Friend 2 $\rightarrow$ [3, 4]

Each friend gets exactly 2 marbles.

Also, no friend gets two marbles of the same type.

Therefore, the answer is:

True

Example 2**Input:**

marbles = [1, 2, 2, 4]
k = 3

Output:

false

Explanation

There are 4 marbles, but every friend must receive exactly 3 marbles.

Since:

$$4 \% 3 \neq 0$$

we cannot divide all 4 marbles into groups of exactly 3.

Therefore, the answer is:

False

Approach to Solve This Problem:

There are two things we need to check.

Step 1: Total number of marbles must be divisible by k

Every friend gets exactly k marbles.

So the total number of marbles must be divided into groups of size k.

Therefore:

$$n \% k == 0$$

must be true.

For example:

$$n = 12$$

$$k = 3$$

is possible because:

$$12 \% 3 = 0$$

There will be:

$$12 / 3 = 4 \text{ friends}$$

But:

$$n = 10$$

$$k = 3$$

is impossible because:

$$10 \% 3 \neq 0$$

Step 2: Check how many times each type occurs

This is the more important condition.

Suppose:

$$\text{marbles} = [1, 1, 2, 2, 3, 3]$$

$$k = 3$$

There are:

3 types: 1, 2, 3

and each type appears twice.

We can distribute them as:

Friend 1 $\rightarrow$ [1, 2, 3]

Friend 2 $\rightarrow$ [1, 2, 3]

This works.

Why?

Because each friend receives each type at most once.

If a type appears x times, we need at least x different friends to distribute those marbles.

Therefore:

The number of times any one type appears cannot be greater than the number of friends.

6. How Many Friends Are Needed?

The number of friends is:

number of friends = n / k

For example:

$n = 12$

$k = 3$

Then:

$12 / 3 = 4$ friends

So no marble type can appear more than 4 times.

If one type appears 5 times, we would need at least 5 friends, but only 4 friends are needed to distribute all the marbles.

That would make the distribution impossible.

Step-by-Step Example

Consider:

marbles = [1, 1, 2, 2, 3, 3]
 $k = 3$

Step 1: Check the total

There are 6 marbles.

$$6 \% 3 = 0$$

So the total can be divided equally.

Number of friends:

$$6 / 3 = 2$$

So we have 2 friends.

Step 2: Count each type

1 $\rightarrow$ 2 times

2 $\rightarrow$ 2 times

3 $\rightarrow$ 2 times

The maximum frequency is 2.

Number of friends is also 2.

So:

maximum frequency $\leq$ number of friends

Therefore, the distribution is possible.

One possible distribution is:

Friend 1 $\rightarrow$ [1, 2, 3]

Friend 2 $\rightarrow$ [1, 2, 3]

Answer:

True

Another Example

Consider:

marbles = [1, 1, 1, 2, 2, 3]

$k = 3$

There are 6 marbles, so:

$$6 / 3 = 2 \text{ friends}$$

But type 1 appears 3 times.

We only have 2 friends.

A friend cannot receive two 1s, so there is no way to distribute the three 1s among only two friends.

Therefore:

false

Java Code:

```
1  import java.util.*;
2
3  public class Main {
4
5      public static boolean canPartition(int[] marbles, int k) {
6
7          int n = marbles.length;
8
9          // Total marbles must be divisible by k
10         if (n % k != 0) {
11             return false;
12         }
13
14         // Number of friends required
15         int friends = n / k;
16
17         // Count frequency of each marble type
18         HashMap<Integer, Integer> frequency = new HashMap<>();
19
20         for (int marble : marbles) {
21             frequency.put(
22                 marble,
23                 frequency.getOrDefault(marble, defaultValue: 0) + 1
24             );
25         }
26
27         // No type can appear more times than the
28         // number of friends
29         for (int count : frequency.values()) {
30             if (count > friends) {
31                 return false;
32             }
33         }
34
35         return true;
36     }
37 }
```

```
3 public class Main {
37
38     Run | Debug
39     public static void main(String[] args) {
40         Scanner sc = new Scanner(System.in);
41
42         int n = sc.nextInt();
43
44         int[] marbles = new int[n];
45
46         for (int i = 0; i < n; i++) {
47             marbles[i] = sc.nextInt();
48         }
49
50         int k = sc.nextInt();
51
52         System.out.println(canPartition(marbles, k));
53
54         sc.close();
55     }
56 }
```

Solution:

The solution is based on two simple checks:

Check 1

The total number of marbles must be divisible by k.

$n \% k == 0$

This tells us whether we can create groups of exactly k marbles.

Check 2

Find the number of friends:

$\text{friends} = n / k$

Then count how many times each marble type occurs.

If any type occurs more than friends times, the answer is false.

Otherwise, the answer is true.

The important observation is:

A marble type that appears x times needs at least x different friends because the same friend cannot receive that type twice.

Understanding the Important Part of the Code:

1. Check Divisibility

```
if (n % k != 0) {  
    return false;  
}
```

Every friend must receive exactly k marbles.

Therefore, the total number of marbles must be divisible by k .

If it isn't, we can immediately return false.

2. Find the Number of Friends

```
int friends = n / k;
```

For example:

```
n = 12  
k = 3
```

Then:

```
friends = 12 / 3 = 4
```

So we need 4 friends.

3. Count Each Marble Type

```
HashMap<Integer, Integer> frequency = new HashMap<>();
```

The HashMap stores:

marble ID $\rightarrow$ number of times it appears

For example:

```
[1, 1, 2, 2, 3]
```

will give approximately:

```
1  $\rightarrow$  2  
2  $\rightarrow$  2  
3  $\rightarrow$  1
```

4. getOrDefault()

```
frequency.put(
```



```
    marble,  
    frequency.getDefault(marble, 0) + 1  
);
```

This simply means:

If we have seen this marble before, increase its count. Otherwise, start its count at 1.

For example, when we see 2 for the first time:

$2 \rightarrow 1$

When we see it again:

$2 \rightarrow 2$

5. Check the Maximum Frequency

```
for (int count : frequency.values()) {  
    if (count > friends) {  
        return false;  
    }  
}
```

If a type appears more times than the number of friends, it is impossible to distribute those marbles.

Why?

Because two marbles of the same type cannot go to the same friend.

So every occurrence of that type needs a different friend.

Complexity Analysis:

Time Complexity: $O(n)$

We go through the array once to count the frequencies and once through the frequency values to check them.

Both operations together take:

Time Complexity = $O(n)$

Space Complexity: $O(n)$

In the worst case, every marble has a different ID, so the HashMap can contain n different types.

Space Complexity = $O(n)$